Policy enforcement in Cloud Deployer Pro

Overview

Cloud Deployer Pro Advanced Multi-Cloud Orchestration now supports policy enforcement in deployment pipelines.

Policy enforcement helps teams apply security and compliance requirements consistently across connected cloud and on-premises environments. It can:

- block noncompliant deployments;
- apply different enforcement behavior in development, staging, and production;
- enforce required resource tags;
- compare the desired resource state with the existing state; and
- detect changes made outside the deployment pipeline.

The policy engine uses a simplified YAML policy definition that compiles to Rego and runs through Open Policy Agent. Automatic remediation remains experimental for some resource types.

Intended audience

This overview is intended for experienced developers who configure Cloud Deployer Pro deployment pipelines and are familiar with YAML, cloud APIs, and resource models.

About this guide

This page is the parent overview for the policy-enforcement feature. It explains the feature, its lifecycle, the artifacts involved, and the relationship between policy definition, deployment enforcement, and drift detection.

Detailed execution guides will be published after the documentation team can:

- access the feature in Cloud Deployer Pro;
- perform each workflow from beginning to end;
- verify the complete UI navigation and field names;
- confirm the YAML schemas and supported values;
- validate CLI commands and outputs;
- test permissions, role assumption, and failure behavior; and
- review the procedures with engineering and security subject-matter experts.

Policy enforcement in the deployment lifecycle

Policy enforcement applies across three phases.

Pre-deployment

Before deployment:

1. Define policy constraints and violation responses in *policy-manifest.yaml*.
2. Add `conditions` blocks when a policy applies only to a specific environment or application type.
3. Define required tags, such as Owner, CostCenter, and Environment.
4. Compose modular policy files into *policy-set.yaml* when several policy domains apply.
5. Reference the policy manifest or policy set from the top-level `policies` block in *pipeline.yml*.
6. Specify applicable stages and configure stage-specific overrides.
7. Run `cdp policy validate`.

Deployment

During deployment:

1. Run `cdp pipeline run --policy-check`.
2. Compare the desired resource state with the existing state.
3. Apply the configured response when a constraint is violated:
   - deny the deployment;
   - warn and log; or
   - automatically remediate the resource when supported.
4. Review verbose violation information and Rego traces when troubleshooting a custom policy.

Post-deployment

After deployment:

1. Monitor deployed resources for changes made outside the pipeline.
2. Run `cdp policy detect-drift --pipeline <id>` for on-demand drift detection.
3. Send drift alerts through webhooks or SNS/SQS integrations.
4. Restore the declared state manually or rerun the pipeline.
5. Keep changes flowing through the deployment pipeline to support the immutability principle.

Core configuration artifacts

policy-manifest.yaml

Defines desired state, policy constraints, violation responses, optional conditions, and required tag rules.

Example policy intents include:

- require approved KMS encryption for Amazon S3 buckets;

- prevent public ingress to databases;
- require Owner, CostCenter, and Environment tags; and
- limit resource sizes by environment.

policy-set.yaml

Combines modular policy files, such as security, compliance, and cost policies, into one reusable policy set.

pipeline.yml

References the policy manifest or policy set from a top-level `policies` block. It identifies applicable stages and can define stage-specific overrides.

For example, development can warn about selected violations while production denies the same violations.

Policy-engine identity

Policy evaluation uses a separate, granular, read-only IAM role or service account.

The source identifies the configuration location as:

Settings > Cloud Integrations > Policy Engine IAM Role

The policy-engine identity:

- can support cross-account or cross-cloud role assumption;
- must remain separate from the deployment identity; and
- requires only the access needed to evaluate existing resource state.

The complete configuration procedure depends on product access and confirmation of provider-specific permissions, fields, and role-assumption behavior.

Enforcement responses

Response	Behavior
Deny	Blocks the deployment when a constraint is violated.
Warn and log	Allows the deployment to continue and records the violation.
Automatically remediate	Attempts to correct a supported resource. This response remains experimental for some resource types.

Drift detection and immutability

The policy engine continues monitoring deployed resources after deployment. It identifies changes made outside the deployment pipeline and can notify operational teams through webhooks or SNS/SQS.

Teams can restore the declared state manually or rerun the pipeline. Routing changes through the pipeline keeps the declared and deployed states aligned and supports the immutability principle.

Related detailed guides

Pre-deployment guides

- **Create and validate a policy manifest** - Coming soon
- **Define constraints, conditions, and required tags** - Coming soon
- **Compose modular policies in a policy set** - Coming soon
- **Reference policies in a deployment pipeline** - Coming soon
- **Configure stage-specific policy behavior** - Coming soon
- **Configure the policy-engine identity** - Coming soon

Deployment guides

- **Run a pipeline with policy checks** - Coming soon
- **Review and resolve policy violations** - Coming soon
- **Interpret custom-policy Rego traces** - Coming soon
- **Use policy enforcement responses** - Coming soon

Post-deployment guides

- **Run on-demand drift detection** - Coming soon
- **Configure webhook or SNS/SQS drift alerts** - Coming soon
- **Review and resolve configuration drift** - Coming soon

  Publication note: Detailed guides will be completed after the workflows can be performed and verified in the product environment. Each guide will include confirmed prerequisites, complete navigation, exact configuration fields, validated commands, expected results, common errors, and SME-reviewed recovery guidance.

Questions for Lena

1. Can you provide the supported schemas and an approved example for *policy-manifest.yaml* and *policy-set.yaml*?
2. What is the exact syntax of the top-level `policies` block in *pipeline.yml*?
3. How are stages, stage-specific responses, and policy parameters configured?

4. What is the complete navigation and configuration workflow for **Policy Engine IAM Role**?
5. What minimum permissions does the policy-engine identity require for each supported cloud provider?
6. How is cross-account or cross-cloud role assumption configured and validated?
7. What is the correct spelling and complete syntax of the command shown as `cdp policy enforc`?
8. Which resource types support automatic remediation, and what happens when remediation fails?
9. What fields appear in policy violations, validation output, and Rego traces?
10. How are webhook and SNS/SQS drift alerts configured?
11. How frequently does continuous drift detection run?
12. What indicates that drift has been resolved successfully?